

Lab 2

Dynamic Memory Allocation

Implementation, dry runs, debugging, complete solutions, and viva preparation for heap memory, 1D arrays, 2D equal rows, and jagged arrays.

Dark Developer Handbook

new / delete

new[] / delete[]

1D Dynamic Array

2D char array

Jagged int array

Stack vs Heap

Built from the uploaded Lab 2 sheet. Source pages show: page 1 lab goals, page 2 given examples for single integer and dynamic array, page 3 the three required programming tasks.

Section 1: Lab Map

What this lab covers: dynamic memory allocation in C++ using `new`, `delete`, `new[]`, and `delete[]`. The lab starts with single heap values, then moves to 1D arrays, equal-row 2D character arrays, and uneven-row 2D integer arrays.

Central skill: controlling memory manually. You do not just create data. You also become responsible for releasing it. The heap is useful, but it is a dragon with a receipt book: every allocation must be paid back correctly.

Lab Component	What You Build	C++ Concepts	Skill Built
Given Example 1	A single integer created on the heap	<code>int*</code> , <code>new int</code> , dereference, <code>delete</code>	Understand pointer-to-one-value memory
Given Example 2	A fixed-size dynamic integer array	<code>new int[size]</code> , loops, indexing, <code>delete[]</code>	Allocate, fill, print, and free a heap array
Task 1	User-sized dynamic 1D integer array	Input size, heap array, loop-based input/output	Build arrays when size is known only at runtime
Task 2	2D character array with equal rows	<code>char**</code> , row pointers, strings, null terminator	Store multiple same-capacity strings
Task 3	2D integer array with uneven rows	<code>int**</code> , row sizes, jagged allocation	Handle rows with different lengths safely

Source page 1

Rendered and inspected from original lab sheet.

Source page 2

Rendered and inspected from original lab sheet.

Source page 3

Rendered and inspected from original lab sheet.

Section 2: Page or Slide Extraction Map

Page	Visible Topic	Extracted Requirements	Unreadable or Ambiguous Content
Page 1	Lab 2 - Dynamic Memory Allocation	Explore dynamic allocation using new and delete. Learn allocation and deallocation for single variables, 1D arrays, and 2D arrays.	No unreadable task content.
Page 2	Dynamic memory examples	Defines new as heap allocation and delete as deallocation. Shows Example 1: allocate one integer, assign 10, print, delete. Shows Example 2: allocate dynamic array of size 5, fill with 1 * 2, print, delete[.].	The examples are readable. The guide adds nullptr after deletion as a safety habit.
Page 3	Programming tasks	Task 1: dynamic 1D integer array. Task 2: dynamic 2D character array with equal rows, strings per row. Task 3: dynamic 2D integer array with uneven rows.	Task 2 does not specify whether strings contain spaces. This guide uses one word per row and protects the row capacity.

Lab Coverage Plan

Task or Topic	Data Structure / Memory Shape	Implementation Plan	Verification
Single integer example	One pointer on stack pointing to one heap integer	Allocate with new int, assign through dereference, print, delete	Expected output: Value at ptr: 10
1D integer array	One stack pointer pointing to contiguous heap cells	Read size, allocate int[size], input loop, print loop, delete[]	Test positive size and normal values; reject invalid size
Equal-row 2D char array	Array of row pointers, each row is columns + 1 chars	Read rows/columns, allocate char**, allocate each row, input each string, print, delete rows then outer array	Test several strings shorter than column limit
Uneven-row 2D int array	Array of row pointers plus separate row-size array	Read rows, read each row length, allocate each row separately, input/output using columns[i], delete all memory	Test row lengths such as 2, 4, 3

SECTION 3

Implementation Toolkit

The control panel before the code: identify the memory shape, choose the pointer type, then plan the delete order.

Section 3: Implementation Toolkit

1. What This Lab Is Really About

This lab is not only about writing `new` and `delete`. It is about learning that runtime-sized data cannot always live comfortably as normal fixed variables. When the size is known only after the program starts, heap memory becomes the storage room.

The main lesson: **allocation creates responsibility**. A pointer remembers where the memory begins. Loops use that memory. A matching delete releases it. If any step is skipped or mismatched, the program becomes a small haunted house of leaks, dangling pointers, or out-of-bounds access.

2. Concepts Before Code

Question Before Coding	Answer for This Lab
What data is needed?	Sizes from the user, then integer values or strings stored in heap memory.
What C++ tools are required?	Pointers, new, delete, new[], delete[], loops, array indexing, and input/output.
What memory is created?	Heap memory for one int, 1D arrays, row pointer arrays, equal-size char rows, and jagged int rows.
What must be deleted?	A single object uses delete. Arrays use delete[]. 2D arrays require deleting every row first, then deleting the outer pointer array.
What can go wrong?	Negative sizes, forgetting delete[], using delete for arrays, writing outside row limits, using one shared column count for jagged rows.

How to Recognize the Coding Pattern

Task Pattern	C++ Tool Needed	First Move	Memory Concern	Common Trap
Single heap integer	int*, new int, delete	Declare pointer and allocate one integer	Use delete, not delete[]	Forgetting to dereference with *ptr
Dynamic 1D array	new[], delete[], loops	Read size, validate it, allocate array	Must free with delete[]	Using invalid index or deleting with plain delete
Dynamic 2D equal rows	char**, row pointers	Allocate row pointer array first	Delete each row, then the outer array	For strings, forgetting space for \\0
Jagged 2D array	int** + row-size array	Store each row length before input/output	Each row has different size	Using the same column count for every row
Pointer cleanup	delete, delete[], nullptr	Release heap memory after last use	Prevent memory leaks and dangling pointers	Using pointer again after deletion

4. Lab Decision Tree

Decision Tree: What Should I Write First?

1. Is the task asking for one value on the heap?
 - Use `T* ptr = new T;`
 - Access with `*ptr`.
 - Release with `delete ptr`.
2. Is the task asking for one row of many values?
 - Read size.
 - Use `T* arr = new T[size];`
 - Use `arr[i]` in loops.
 - Release with `delete[] arr`.
3. Is the task asking for a 2D array with equal row size?
 - Read rows and columns.
 - Use `T** data = new T*[rows];`
 - For each row, allocate new `T[columns]`.
 - Delete each row, then `delete[] data`.
4. Is the task asking for uneven or jagged rows?
 - Read rows.
 - Keep `int* columns = new int[rows];`
 - Allocate each row using `columns[i]`.
 - Input, print, and delete using `columns[i]`.
5. Is the task storing C-style strings?
 - Allocate `columns + 1` characters per row.
 - The extra cell stores the null terminator `'\0'`.

Required C++ Prerequisite Bridge

Pointer Translation

`int* ptr` means the variable `ptr` stores the address of an integer. `*ptr` means the integer value stored at that address.

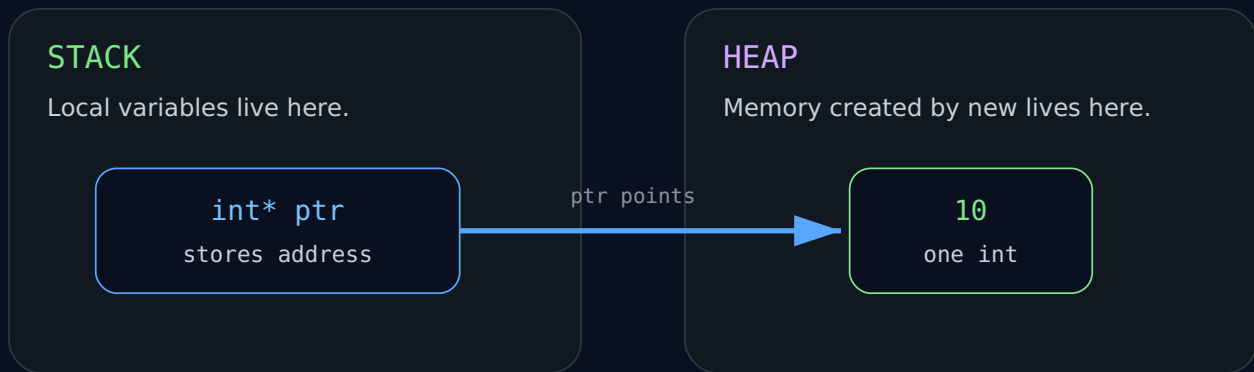
Array Translation

`arr[i]` means move `i` cells from the beginning of the array and access that element. In dynamic arrays, this only works safely if $0 \leq i < \text{size}$.

Section 4: Concept and Implementation Foundation

Stack vs Heap

Memory Diagram: pointer on stack, value on heap



Memory Area	What Lives There	Who Manages It	Lab Meaning
Stack	Local variables like <code>int size</code> and pointer variables like <code>int* arr</code>	Mostly automatic	Pointer variables disappear when the function ends.
Heap	Memory created by <code>new</code> or <code>new[]</code>	You, the programmer	Heap memory stays until you delete it.
Danger Zone	Memory that was allocated but never deleted	Nobody, after you forget it	This is a memory leak. The program loses the address.
Dangling Pointer	Pointer still holding an address after the memory was deleted	Unsafe	Set pointer to <code>nullptr</code> after delete when practical.

Given Example 1: Single Dynamic Integer

● ● ● Cleaned Example 1 - single heap integer

```
#include <iostream>
using namespace std;

int main() {
    int* ptr = new int;
    *ptr = 10;
    cout << "Value at ptr: " << *ptr << endl;
    delete ptr;
    ptr = nullptr;
    return 0;
}
```

Expected Output

Value at ptr: 10

What this example teaches

`new int` allocates one integer on the heap. The pointer `ptr` stores its address. `*ptr = 10` writes into that heap location. `delete ptr` releases exactly one integer.

Given Example 2: Dynamic Array

● ● ● Cleaned Example 2 - dynamic integer array

```
#include <iostream>
using namespace std;

int main() {
    int size = 5;
    int* arr = new int[size];

    for (int i = 0; i < size; i++) {
        arr[i] = i * 2;
    }

    cout << "Array elements: ";
    for (int i = 0; i < size; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;

    delete[] arr;
    arr = nullptr;

    return 0;
}
```

Expected Output

Array elements: 0 2 4 6 8

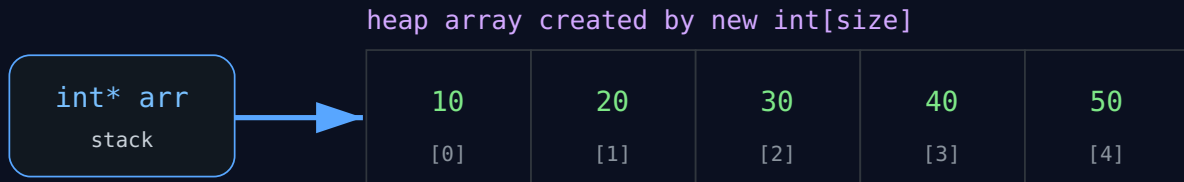
Correction Habit Added

The lab example is conceptually correct. This guide adds `arr = nullptr;` after `delete[] arr;`. It is not required for this tiny program, but it teaches a safe pattern: after memory is released, avoid leaving a pointer that still looks usable.

Section 5: Algorithm Workflows

Workflow A: Allocate and Delete a 1D Array

1D Dynamic Array Shape



One `new[]` needs one matching `delete[]`.

Algorithm: Dynamic 1D Integer Array

Input: size and size integer values

Output: printed array values

1. Read size.
2. If size ≤ 0 , stop with an error message.
3. Allocate `arr = new int[size]`.
4. For i from 0 to size - 1, read `arr[i]`.
5. For i from 0 to size - 1, print `arr[i]`.
6. Release memory using `delete[] arr`.
7. Optionally set `arr = nullptr`.

Workflow B: Allocate and Delete a 2D Equal-Row Character Array

2D Equal-Row Character Array Shape

`char** words`: row pointer array + equal-length char rows
columns + 1 chars per row



Delete each row first, then `delete[] words`.

Algorithm: 2D Char Array With Equal Rows

Input: rows, columns, and one string for each row

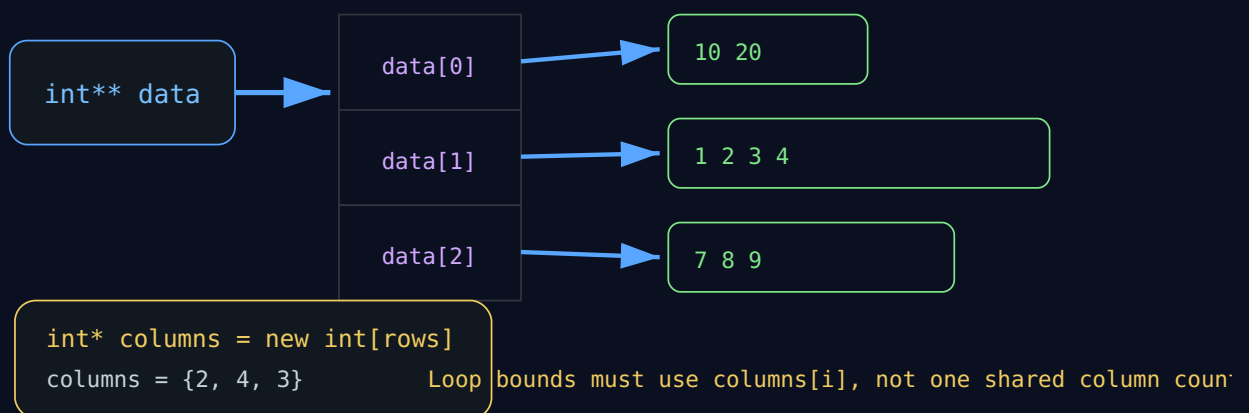
Output: all stored strings

1. Read rows and columns.
2. Allocate words = new char*[rows].
3. For each row i, allocate words[i] = new char[columns + 1].
4. Read one string into each row. Limit input to columns characters.
5. Print every row as a C-style string.
6. For each row i, delete[] words[i].
7. Delete[] words.

Workflow C: Allocate and Delete a Jagged 2D Integer Array

Jagged 2D Integer Array Shape

Jagged int**: each row can have a different length



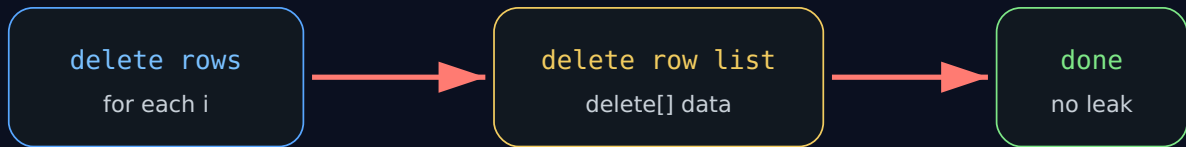
Algorithm: Jagged 2D Integer Array

Input: rows, each row length, and row values

Output: all row values

1. Read rows.
2. Allocate columns = new int[rows].
3. Allocate data = new int*[rows].
4. For each row i:
 - a. Read columns[i].
 - b. Allocate data[i] = new int[columns[i]].
5. For each row i and column j, read data[i][j].
6. For each row i and column j, print data[i][j].
7. Delete every data[i].
8. Delete[] data.
9. Delete[] columns.

Delete Order for 2D Arrays



Never delete only the outer pointer if inner rows were allocated separately.

SECTION 6

Complete Lab Task Solutions

Every required task, expanded into plan, code, sample run, trace, edge cases, complexity, debugging, and viva answers.

Task 1: Dynamic Array Allocation (1D)

Task statement: Use `new` to allocate an integer array of user-specified size. Input each element, print the values, and deallocate the array using `delete[]`.

What You Need to Know First

Need	Reason
A runtime size	The user decides the array size while the program is running.
<code>int* arr</code>	A pointer is needed to remember the first heap cell.
Input and output loops	The program must visit every cell exactly once for input and printing.
<code>delete[] arr</code>	The array was created with <code>new[]</code> , so it must be freed with <code>delete[]</code> .

Implementation Plan

1. Read size from the user.
2. Reject non-positive size.
3. Allocate `int* arr = new int[size];`.
4. Use a for loop to input values.
5. Use another for loop to print values.
6. Free memory with `delete[] arr;` and set pointer to `nullptr`.

Pseudocode: Task 1

```
READ size
IF size <= 0 THEN
    PRINT error message
    STOP
END IF
arr <- new integer array of length size
FOR i <- 0 to size - 1
    READ arr[i]
END FOR
FOR i <- 0 to size - 1
    PRINT arr[i]
END FOR
delete[] arr
arr <- nullptr
```

Task 1 Solution - task1_dynamic_1d.cpp

```
#include <iostream>
using namespace std;

int main() {
    int size;

    cout << "Enter array size: ";
    cin >> size;

    if (size <= 0) {
        cout << "Size must be positive." << endl;
    }
}
```

```

    return 0;
}

int* arr = new int[size];

cout << "Enter " << size << " integer values: ";
for (int i = 0; i < size; i++) {
    cin >> arr[i];
}

cout << "Array values: ";
for (int i = 0; i < size; i++) {
    cout << arr[i] << " ";
}
cout << endl;

delete[] arr;
arr = nullptr;

return 0;
}

```

Sample Run

```

Enter array size: 5
Enter 5 integer values: 10 20 30 40 50
Array values: 10 20 30 40 50

```

Code Explanation

Code Part	Purpose	What to Watch
<code>int* arr = new int[size];</code>	Creates size integers on the heap.	Only valid if size is positive.
Input loop	Stores values into <code>arr[0]</code> through <code>arr[size-1]</code> .	Do not use <code>i <= size</code> ; that goes out of bounds.
Output loop	Prints each stored value.	Use the same valid index range.
<code>delete[] arr;</code>	Releases the heap array.	Must match <code>new[]</code> .

Dry Run

Step	Variable State	Action	Output
1	<code>size = 5</code>	Read user size	
2	<code>arr -> heap array of 5 int cells</code>	Allocate memory	
3	<code>arr = {10,20,30,40,50}</code>	Input loop stores values	
4	Same values	Output loop prints values	Array values: 10 20 30 40 50
5	<code>arr = nullptr</code>	Delete heap memory	

Edge Cases, Common Bugs, Complexity

	Expected Handling	Bug	Fix
--	-------------------	-----	-----

Edge Case		Using <code>delete arr</code>	Use <code>delete[] arr</code> .
<code>size = 1</code>	Allocate one integer and print one value.	Loop uses <code>i <= size</code>	Use <code>i < size</code> .
<code>size <= 0</code>	Show error and stop.	Printing before input	Input values first.
Large size	May fail if memory is not available; beginner labs usually use small sizes.		

Operation	Time Complexity	Space Complexity
Input all values	$O(n)$	$O(n)$ heap memory
Print all values	$O(n)$	No extra significant memory
Delete array	$O(1)$ from code perspective	Frees $O(n)$ heap memory

Viva Questions - Task 1

1. **Why use dynamic allocation?** Because the array size is decided at runtime.
2. **Why `delete[]`?** Because the memory was created using `new[]`.
3. **What is a memory leak?** Allocated memory that is never deallocated.
4. **What happens if index equals size?** It accesses outside the array and causes undefined behavior.

Task 2: Dynamic Array Allocation (2D) with Equal Rows

Task statement: Allocate a two-dimensional character array where the number of rows and columns is provided by the user. Input one string for each row, print all strings, and deallocate the memory.

What You Need to Know First

Need	Reason
<code>char** words</code>	A 2D dynamic array is usually represented as a pointer to row pointers.
Rows and columns	Rows decide how many strings. Columns decide max characters per string.
<code>columns + 1</code>	C-style strings need one extra cell for the null terminator <code>\0</code> .
Delete order	Delete each row first, then delete the outer row pointer array.

String Capacity Rule

If the user says `columns = 8`, the code allocates 9 chars per row. The extra character is for `\0`, which marks the end of a C-style string. Without it, printing strings can wander into memory fog.

Implementation Plan

1. Read rows and columns.
2. Validate both are positive.
3. Allocate row pointers using `char** words = new char*[rows];`.
4. For every row, allocate new `char[columns + 1]`.
5. Read one word/string per row with width protection.
6. Print every stored row.
7. Delete every row, then delete the outer pointer array.

Pseudocode: Task 2

```
READ rows, columns
IF rows <= 0 OR columns <= 0 THEN STOP
words <- new char pointer array of length rows
FOR i <- 0 to rows - 1
    words[i] <- new char array of length columns + 1
END FOR
FOR i <- 0 to rows - 1
    READ words[i] with maximum columns characters
END FOR
FOR i <- 0 to rows - 1
    PRINT words[i]
END FOR
```



```

FOR i <- 0 to rows - 1
    delete[] words[i]
END FOR
delete[] words

```

● ● ● Task 2 Solution - task2_dynamic_2d_char_equal.cpp

```

#include <iostream>
#include <iomanip>
#include <limits>
using namespace std;

int main() {
    int rows, columns;

    cout << "Enter number of rows: ";
    cin >> rows;
    cout << "Enter maximum characters per row: ";
    cin >> columns;

    if (rows <= 0 || columns <= 0) {
        cout << "Rows and columns must be positive." << endl;
        return 0;
    }

    char** words = new char*[rows];
    for (int i = 0; i < rows; i++) {
        words[i] = new char[columns + 1];
    }

    cout << "Enter one string for each row." << endl;
    cout << "Each string can contain at most " << columns << " characters." << endl;

    for (int i = 0; i < rows; i++) {
        cout << "Row " << i << ": ";
        cin >> setw(columns + 1) >> words[i];
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }

    cout << "Stored strings:" << endl;
    for (int i = 0; i < rows; i++) {
        cout << words[i] << endl;
    }

    for (int i = 0; i < rows; i++) {
        delete[] words[i];
    }
    delete[] words;
    words = nullptr;

    return 0;
}

```

Sample Run

```

Enter number of rows: 3
Enter maximum characters per row: 8
Enter one string for each row.
Each string can contain at most 8 characters.
Row 0: alpha
Row 1: beta
Row 2: gamma
Stored strings:
alpha
beta
gamma

```

Code Explanation

Code Part	Purpose	What to Watch
<code>char** words = new char*[rows];</code>	Creates the outer array of row pointers.	This does not create the actual character rows yet.
<code>words[i] = new char[columns + 1];</code>	Creates one string row.	The +1 is for the null terminator.
<code>setw(columns + 1)</code>	Limits input so it does not overflow the row.	Requires <code><iomanip></code> .
Delete loop	Releases every row.	Must happen before deleting words.
<code>delete[] words;</code>	Releases the row pointer array.	Do this after all rows are deleted.

Dry Run

Row	Allocated Capacity	Input	Stored String
0	9 chars	alpha	a l p h a \0
1	9 chars	beta	b e t a \0
2	9 chars	gamma	g a m m a \0

Edge Cases, Common Bugs, Complexity

Edge Case	Expected Handling	Bug	Fix
Rows = 1	Works as one string row.	Allocating new <code>char[columns]</code> only	Use <code>columns + 1</code> for C-style strings.
Columns = 1	Allows one-character strings plus <code>\0</code> .	Deleting only <code>delete[] words</code>	Delete every <code>words[i]</code> first.
Input longer than columns	This solution limits the read to avoid buffer overflow.	Using one <code>char*</code> for all rows	Allocate each row separately.
Strings with spaces	This solution reads one word per row; use <code>getline</code> if full lines are required.		

Operation	Time Complexity	Space Complexity
Allocate rows	$O(r)$	$O(r)$ row pointers
Allocate all char rows	$O(r)$	$O(r * c)$ chars
Input/print strings	$O(r * c)$ worst case	Uses allocated 2D memory
Delete all memory	$O(r)$	Frees $O(r * c)$ memory

Viva Questions - Task 2

1. **Why use `char**`?** It stores addresses of multiple character rows.
2. **Why allocate `columns + 1`?** C-style strings need space for the null terminator.
3. **How do you delete this 2D array?** Delete every row with `delete[] words[i]`, then delete the outer pointer array.
4. **What happens if you forget to delete rows?** The rows leak even if the outer pointer array is deleted.

Task 3: Dynamic Array Allocation (2D) with Uneven Rows

Task statement: Allocate a two-dimensional integer array where each row has a different number of columns. Ask for the number of elements in each row, input values, print the entire 2D array, and deallocate all memory.

What You Need to Know First

Need	Reason
A row-size array	A jagged array cannot use one common column count. Store <code>columns[i]</code> for every row.
<code>int** data</code>	The outer pointer stores addresses of rows.
Nested loops with different limits	The inner loop must use <code>j < columns[i]</code> .
Careful cleanup	Every row was allocated separately, so every row must be deleted separately.

Implementation Plan

1. Read number of rows.
2. Allocate `columns` to store each row length.
3. Allocate `data` to store row pointers.
4. For each row, read its length and allocate that exact number of integers.
5. Input row values using `columns[i]` as the loop limit.
6. Print row values using the same row-specific limit.
7. Delete each row, then delete `data` and `columns`.

Pseudocode: Task 3

```
READ rows
IF rows <= 0 THEN STOP
columns <- new int array of length rows
data <- new int pointer array of length rows
FOR i <- 0 to rows - 1
    READ columns[i]
    IF columns[i] <= 0 THEN clean up and STOP
    data[i] <- new int array of length columns[i]
END FOR
FOR i <- 0 to rows - 1
    FOR j <- 0 to columns[i] - 1
        READ data[i][j]
    END FOR
END FOR
FOR i <- 0 to rows - 1
    FOR j <- 0 to columns[i] - 1
        PRINT data[i][j]
    END FOR
END FOR
FOR i <- 0 to rows - 1
    delete[] data[i]
```

```
END FOR
delete[] data
delete[] columns
```

● ● ● Task 3 Solution - task3_dynamic_2d_jagged_int.cpp

```
#include <iostream>
using namespace std;

int main() {
    int rows;

    cout << "Enter number of rows: ";
    cin >> rows;

    if (rows <= 0) {
        cout << "Rows must be positive." << endl;
        return 0;
    }

    int* columns = new int[rows];
    int** data = new int*[rows];

    for (int i = 0; i < rows; i++) {
        cout << "Enter number of elements in row " << i << ": ";
        cin >> columns[i];

        if (columns[i] <= 0) {
            cout << "Column count must be positive." << endl;

            for (int j = 0; j < i; j++) {
                delete[] data[j];
            }
            delete[] data;
            delete[] columns;
            return 0;
        }

        data[i] = new int[columns[i]];
    }

    for (int i = 0; i < rows; i++) {
        cout << "Enter " << columns[i] << " values for row " << i << ": ";
        for (int j = 0; j < columns[i]; j++) {
            cin >> data[i][j];
        }
    }

    cout << "Jagged 2D array:" << endl;
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < columns[i]; j++) {
            cout << data[i][j] << " ";
        }
        cout << endl;
    }

    for (int i = 0; i < rows; i++) {
        delete[] data[i];
    }
    delete[] data;
    delete[] columns;

    data = nullptr;
    columns = nullptr;

    return 0;
}
```

Sample Run

```

Enter number of rows: 3
Enter number of elements in row 0: 2
Enter number of elements in row 1: 4
Enter number of elements in row 2: 3
Enter 2 values for row 0: 10 20
Enter 4 values for row 1: 1 2 3 4
Enter 3 values for row 2: 7 8 9
Jagged 2D array:
10 20
1 2 3 4
7 8 9

```

Code Explanation

Code Part	Purpose	What to Watch
<code>int* columns = new int[rows];</code>	Stores the length of each row.	Without it, printing/deleting logic loses row sizes.
<code>int** data = new int*[rows];</code>	Stores row addresses.	Rows are not allocated yet.
<code>data[i] = new int[columns[i]];</code>	Allocates the exact number of cells for row <code>i</code> .	Each row may have a different length.
<code>j < columns[i]</code>	Uses the correct limit for each row.	This is the heart of jagged-array safety.
Cleanup on invalid column count	Deletes already allocated rows before stopping.	Prevents leaks if error happens halfway.

Dry Run

Row	columns[i]	Allocated Values	Printed Output
0	2	10, 20	10 20
1	4	1, 2, 3, 4	1 2 3 4
2	3	7, 8, 9	7 8 9

Pointer	Points To	Status After Deletion
<code>data[0]</code>	Heap row of 2 integers	Released by <code>delete[] data[0]</code>
<code>data[1]</code>	Heap row of 4 integers	Released by <code>delete[] data[1]</code>
<code>data[2]</code>	Heap row of 3 integers	Released by <code>delete[] data[2]</code>
<code>data</code>	Heap array of row pointers	Released by <code>delete[] data</code>
<code>columns</code>	Heap array of row lengths	Released by <code>delete[] columns</code>

Edge Cases, Common Bugs, Complexity

Edge Case	Expected Handling	Bug	Fix
-----------	-------------------	-----	-----

Rows = 1	A jagged array can have one row.	Printing with <code>j < columns</code>	Use <code>j < columns[i]</code> .
A row length = 1	Allocate one value for that row.	Deleting only data	Delete every row first.
Invalid row length	Clean up already allocated memory and stop.	Forgetting columns	Store row lengths in a separate array.
Rows with very different lengths	Works because every row uses <code>columns[i]</code> .	Lost row pointer	Never overwrite <code>data[i]</code> after allocating it.

Operation	Time Complexity	Space Complexity
Read row sizes	$O(r)$	$O(r)$ for row-size array
Input all values	$O(\text{total elements})$	$O(\text{total elements})$ heap memory
Print all values	$O(\text{total elements})$	No extra significant memory
Delete all rows	$O(r)$	Frees all heap rows

Viva Questions - Task 3

1. **What is a jagged array?** A 2D array where rows can have different lengths.
2. **Why store row sizes?** Each row has its own column count, so loops need `columns[i]`.
3. **How is a jagged array deleted?** Delete each row first, then delete the outer pointer array and the row-size array.
4. **What is the biggest common mistake?** Assuming all rows have the same number of columns.

Section 7: Debugging and Common Errors

Symptom	Likely Cause	Fix
Program crashes after input	Index went outside allocated range	Check every loop condition: use <code>i < size</code> and <code>j < columns[i]</code> .
Garbage output	Value printed before being initialized	Input into every allocated cell before printing.
Memory leak	Allocated memory was not deleted	Match every <code>new</code> with <code>delete</code> and every <code>new[]</code> with <code>delete[]</code> .
Crash during 2D cleanup	Deleting outer pointer first	Delete each inner row before deleting the outer pointer array.
String row overflows	Input longer than allocated character capacity	Allocate <code>columns + 1</code> and limit input with <code>setw(columns + 1)</code> .
Jagged row prints too many values	Using shared column count instead of row-specific length	Use <code>columns[i]</code> for every row.
Dangling pointer bug	Using pointer after delete	Do not access after deletion; set pointer to <code>nullptr</code> when practical.

The Delete Pairing Rule

`new int` pairs with `delete`. `new int[size]` pairs with `delete[]`. Mixing them is undefined behavior. Undefined behavior is not a normal error message. It is the compiler goblin saying, "I owe you nothing."

Section 8: Testing Checklist

Done?	Test	Why It Matters
☐	Task 1 with size 1	Checks smallest valid 1D allocation.
☐	Task 1 with size 5 and normal values	Checks normal loop behavior.
☐	Task 1 with size 0 or negative	Checks validation path.
☐	Task 2 with 1 row and small column count	Checks minimal 2D char allocation.
☐	Task 2 with multiple rows	Checks row allocation and printing.
☐	Task 2 with string length equal to max columns	Checks null terminator handling.
☐	Task 3 with row sizes 2, 4, 3	Checks uneven row behavior.
☐	Task 3 with one row only	Checks jagged edge case.
☐	Task 3 with invalid row length	Checks cleanup before early exit.
☐	Run with compiler warnings enabled	Catches suspicious code before viva day.

Section 9: Complexity Summary

Structure / Task	Primary Operations	Time Complexity	Space Complexity
Single heap integer	Allocate, assign, print, delete	$O(1)$	$O(1)$
1D dynamic int array	Input and print n values	$O(n)$	$O(n)$
2D equal-row char array	Input/print r strings of max length c	$O(r * c)$ worst case	$O(r * c)$ plus $O(r)$ row pointers
2D jagged int array	Input/print all row elements	$O(\text{total elements})$	$O(\text{total elements})$ plus $O(r)$ row pointers and row sizes
2D deletion	Delete every row, then outer pointer	$O(r)$	Frees allocated heap memory

Complexity Translation

For jagged arrays, do not force everything into rows * columns. There is no single shared column count. Use **total elements**, meaning the sum of all row lengths.

Section 10: Quick Revision Sheet

Key Syntax Patterns

● ● ● Pointer and Dynamic Memory Cheat Sheet

```
// Single heap value
int* ptr = new int;
*ptr = 10;
delete ptr;
ptr = nullptr;

// 1D heap array
int* arr = new int[size];
delete[] arr;
arr = nullptr;

// 2D equal-row array
int** data = new int*[rows];
for (int i = 0; i < rows; i++) {
    data[i] = new int[columns];
}
for (int i = 0; i < rows; i++) {
    delete[] data[i];
}
delete[] data;

// Jagged row loop
for (int i = 0; i < rows; i++) {
    for (int j = 0; j < columns[i]; j++) {
        cout << data[i][j] << " ";
    }
}
```

Pointer Safety Rules

1. Every new needs one delete.
2. Every new[] needs one delete[].
3. Never access an array outside its valid index range.
4. For 2D dynamic arrays, delete inner rows before deleting the outer pointer array.
5. For C-style strings, allocate one extra character for `\\0`.
6. For jagged arrays, store each row size and use it in loops.

Common Viva Answers

Question	Short Answer
What is dynamic memory allocation?	Creating memory during runtime, usually on the heap.
Why use new?	To allocate heap memory when size or lifetime needs runtime control.
Difference between delete and delete[]?	delete frees one object; delete[] frees an array.
What is a memory leak?	Allocated memory that is never released.
What is a dangling pointer?	A pointer that still stores the address of memory that has already been freed.

What is a jagged array?

A 2D array where each row can have a different number of columns.

Viva Flash Cards - Memory Safety Mini Sheet

Allocation Pairing

- `new int` -> `delete`
- `new int[n]` -> `delete[]`
- Never mix the two.

2D Delete Order

- Delete every row first.
- Then delete the row pointer
- Outer-only delete leaks rows

Jagged Array Rule

- Store `columns[i]` for each row.
- Loop with `j < columns[i]`.
- Total work = sum of row sizes.

Exam instinct: draw the memory shape first, then write the delete order.

Section 11: Lab Coverage Audit

Page / Slide Range	Task or Topic Found	Where It Was Covered in the Guide	Status	Notes
Page 1	Lab title: Dynamic Memory Allocation; goals for new, delete, single variables, 1D and 2D arrays	Sections 1, 3, 4	Explained	No unreadable content.
Page 2	Definition of new and delete; Example 1 single integer; Example 2 dynamic array	Sections 4 and 5	Explained, Corrected, Traced	Added nullptr safety habit after deletion.
Page 3 - Task 1	Dynamic 1D integer array: allocate, input, print, deallocate	Section 6 Task 1	Implemented, Traced, Tested	Code compiled successfully.
Page 3 - Task 2	Dynamic 2D character array with equal rows: input strings, print, deallocate	Section 6 Task 2	Implemented, Traced, Tested	Uses columns + 1 for null terminator and protected input width.
Page 3 - Task 3	Dynamic 2D integer array with uneven rows: row-specific columns, input, print, deallocate	Section 6 Task 3	Implemented, Traced, Tested	Uses row-size array and cleanup on invalid row length.

Final Validation Notes

The submitted programs were compiled with `g++ -std=c++11 -Wall -Wextra -pedantic`. Sample runs were executed for all three tasks. The PDF was rendered for layout inspection after generation.